

Project #3. Key-Value Interface Enablement and KV-SSD Firmware Implementation

OBJECTIVE

Modern storage systems are willing to adopt alternative abstractions beyond the traditional block interface. Among these emerging models, the *Key-Value (KV) interface* has gained attention as a next-generation storage abstraction. A device that supports this interface is referred to as a *Key-Value SSD (KV-SSD)* [1], where data is accessed directly by user-defined keys rather than fixed-size logical block addresses. The KV interface eliminates the need for filesystem-level indexing structures and reduces data movement by removing unnecessary block translations. Furthermore, it shifts part of the key-value management logic from the host software stack into the SSD controller, simplifying host-side software and enabling hardware-accelerated key-value operations.

In this project, you will explore how the Cosmos+ OpenSSD platform can be extended to support the KV interface. Building on your understanding of the existing firmware stack, you will enable the KV command path, implement the necessary firmware routines to store and retrieve key-value pairs, and integrate them into the device's I/O processing pipeline. By the end of this assignment, you will have developed a minimal yet functional KV-SSD firmware that can correctly process **PUT** and **GET** operations issued from the host, and you will validate its correctness through a provided benchmarking tool.

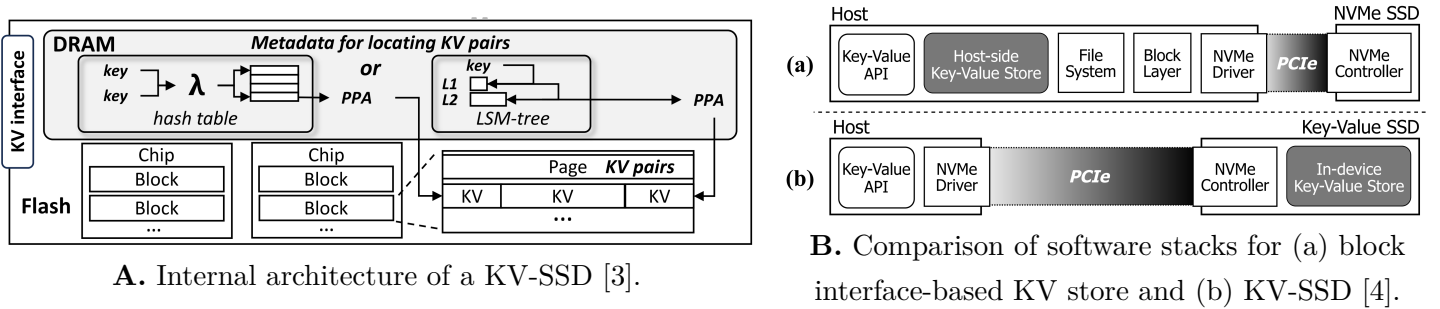


FIGURE 1. Overview of the KV-SSD architecture and its software stack integration.

Duration. May 28 (Thu) – June 11 (Thu) (2 weeks)

TASK DESCRIPTION

Task 1 : Enabling the Key-Value Interface and Developing KV-SSD Firmware.

- Study the provided host-side `kv_bench` program and analyze how it issues NVMe KV commands through the `nvme_passthru.{h|cc}` interfaces. Specifically, understand the exact command format adopted in this assignment and how keys and values are transferred over the NVMe I/O path.
- Based on the KV command format, design and implement the corresponding KV handlers inside the Cosmos+ firmware. (**Hint:** revisit your implementation from Project #1 and recall how custom commands were registered and dispatched in the firmware.)
- Once the interface is enabled, construct an in-device indexing structure for KV storage and lookups. The choice of data structure is entirely up to you (e.g., hash table, LSM-tree, B⁺-tree, or any hybrid design), and you may freely combine in-memory and NAND-backed components as needed.
- Design the mechanism for storing and managing values on NAND flash. Think carefully about how the firmware should allocate, track, and update value locations across KV **PUT** and **GET** operations.

- The following is one possible development direction (**optional**), but recommended if you are unsure where to begin:
 - Implement the index fully in memory for the initial version. Before allocating memory, review `memory_map.h` to understand how physical memory is organized in the OpenSSD environment. Since the system has no operating system supported, all memory must be statically reserved and manually managed.
 - For value storage, recall the fundamental purpose of the Flash Translation Layer (FTL): *to provide a logical address space backed by NAND physical blocks to users*. The firmware itself is also a *user* of that logical space. A simple design is to maintain **an append-only write log in the logical address space**, where each PUT appends a new value and the index tracks its current location.

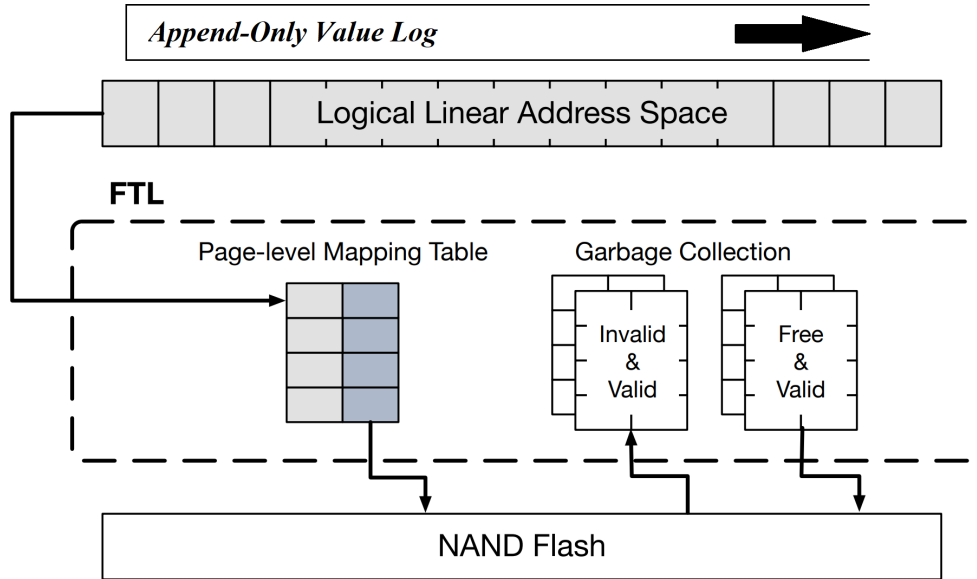


FIGURE 2. Illustration of **an append-only write log** on top of the FTL logical space [2].

Checklist.

- You may freely modify **any firmware source file** under `GreedyFTL/cosmos_app/src/`, and you may also create new source/header files if needed.
- Your firmware must correctly expose the KV interface to the host:
 - The provided host benchmark program (`kv_bench`) should successfully issue NVMe KV-PUT and KV-GET operations using the passthrough interface.
 - Your firmware must decode the NVMe KV command format adopted in this assignment and invoke the appropriate KV handlers without causing device aborts or assertion failures.
- Your KV-SSD firmware must implement the minimum functionality required for correctness:
 - A working PUT path that stores the value and updates an index structure.
 - A working GET path that retrieves the most recent value for a given key.
- To pass this task, the host benchmark must:
 - Start and complete without device aborts or firmware crashes.
 - Successfully detect “No such key” semantics for a known absent key.
 - Print a result summary such as `result: OK=... FAIL=... NO-SUCH-KEY=... elapsed=...` following the expected output format described in the `README.md`.
- Data mismatches (**FAIL**) are **penalized**.

MATERIALS PROVIDED

- **Host-side evaluation program** (prj3-host-evaluation/): Includes `Makefile`, `kv_bench.cc`, `nvme_passthru.(h|cc)`, and `README.md`. Running this binary verifies your firmware by issuing random KV I/O workloads.

STUDENT SUBMISSION

Submit the following items under the naming convention below:

- 1) **workspace/GreedyFTL/cosmos_app/src/ directory** (e.g., `prj3-ssd-{studentID}.tar.gz`)
- 2) **Project Report (exported to PDF)** (e.g., `prj3-document-{studentID}.pdf`)

Submission Window. Opens: May 28 (Thu) 15:00 Closes: Jun 11 (Thu) 23:59

EVALUATION CRITERIA

- **Functionality (60%)** — Firmware must successfully pass evaluations using `kv_bench`. The benchmark must complete without crashes or hangs, and the final result must report OK for all operations with no FAIL entries.
 - Keep in mind that runs outside the default parameters will also be used to to evaluate your work. The default parameters use a key-space size of 4096, with 10000 number of operations (10000, 4096). We will be using a maximum value of (1000000, 4194304) for the parameters.
- **Report Quality (40%)** — clarity, explanation of design, limitations, and improvement plans.

NOTES

- Use administrative privileges (`sudo`) only when absolutely necessary, and always double-check before executing commands.
- Handle the Cosmos+ OpenSSD board with care. Physical mishandling can permanently damage the hardware.
- Any destructive damage to the board or the Linux development machine — whether physical or software-induced — is entirely your responsibility. Such incidents may delay your progress, but all delays and losses during project execution must be borne by you alone.

REFERENCES

- [1] Yanqin Jin, Hung-Wei Tseng, Yannis Papakonstantinou, and Steven Swanson. Kaml: A flexible, high-performance key-value ssd. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 373–384, 2017.
- [2] Chang-Gyu Lee, Hyeongu Kang, Donggyu Park, Sungyong Park, Youngjae Kim, Jungki Noh, Woosuk Chung, and Kyoung Park. ilsm-ssd: An intelligent lsm-tree based key-value ssd for data analytics. In *2019 IEEE 27th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, pages 384–395, 2019.
- [3] Chanyoung Park, Jungho Lee, Chun-Yi Liu, Kyungtae Kang, Mahmut Taylan Kandemir, and Wonil Choi. Anykey: A key-value ssd for all workload types. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1, ASPLOS '25*, page 47–63, New York, NY, USA, 2025. Association for Computing Machinery.
- [4] Junhyeok Park, Chang-Gyu Lee, Soon Hwang, Soonyeal Yang, Jungki Noh, Woosuk Chung, Junghee Lee, and Youngjae Kim. Bandslim: A novel bandwidth and space-efficient kv-ssd with an escape-from-block approach. In *Proceedings of the 53rd International Conference on Parallel Processing, ICPP '24*, page 1187–1196, New York, NY, USA, 2024. Association for Computing Machinery.